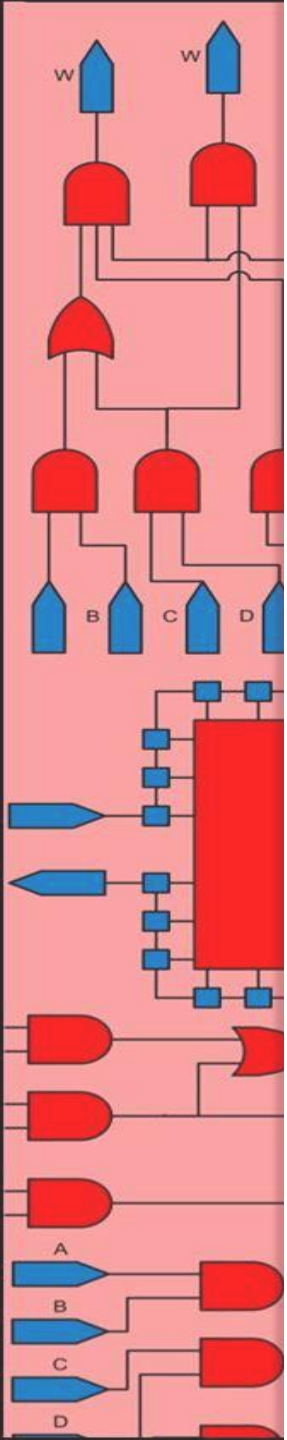


Topic 4

Controllers



Outline

4- Controllers

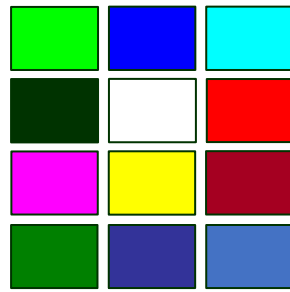
Finite State Machines

4-1 Moore machine

4-2 Mealy machine

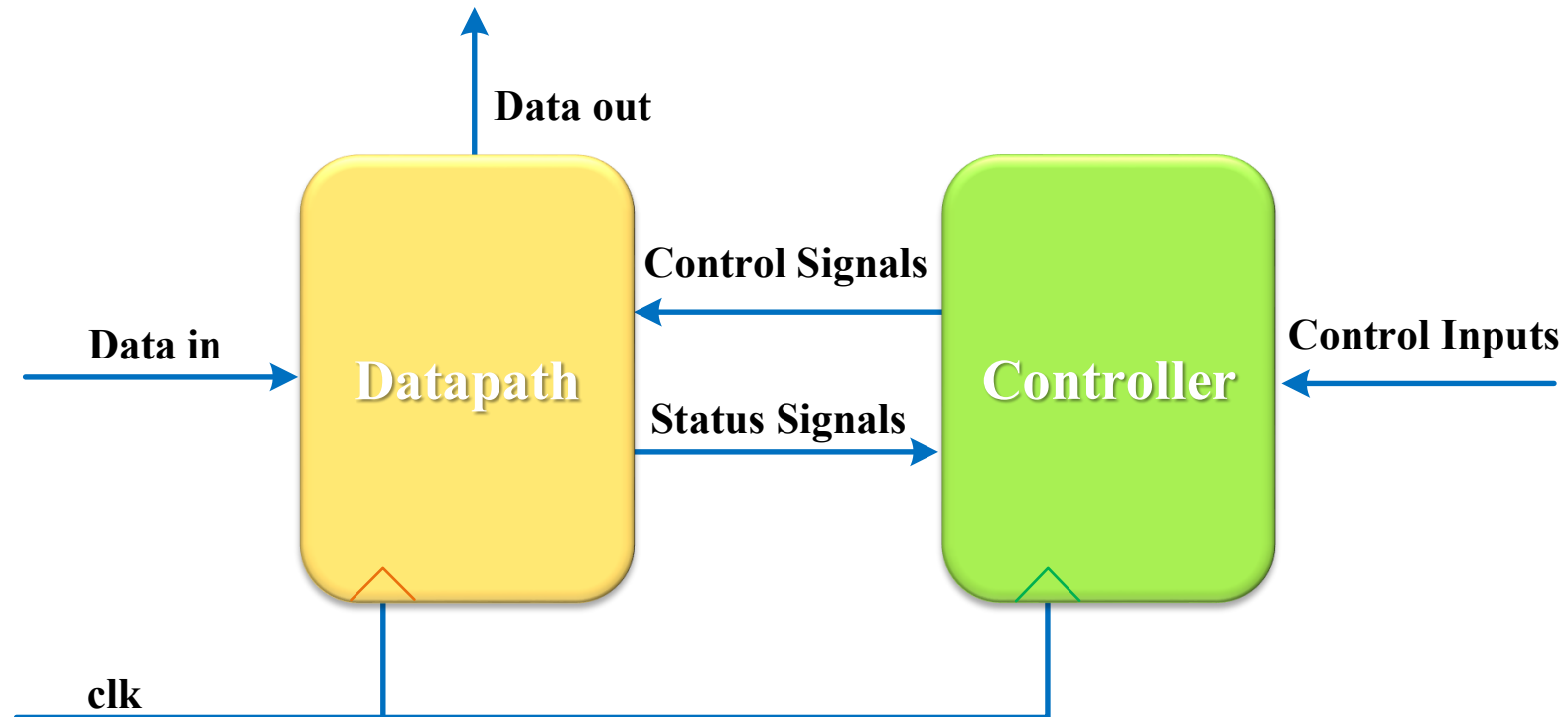
4-3 Huffman model

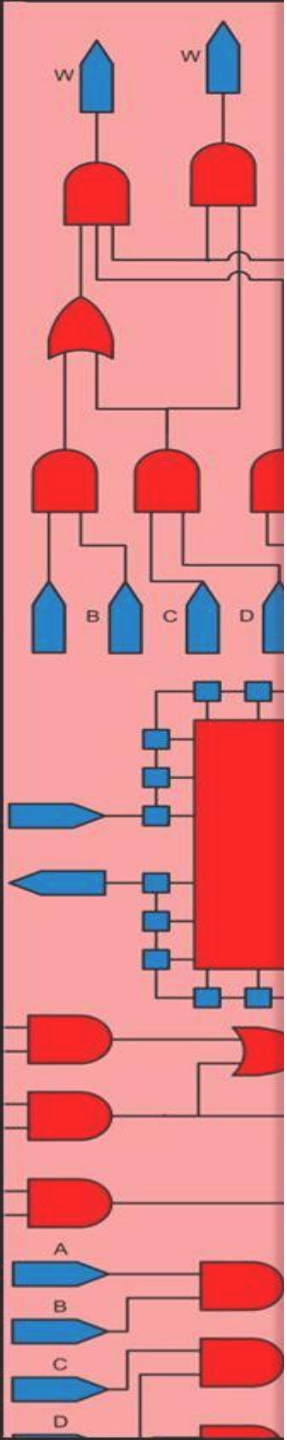
4-4 Hierarchical FSMs



Controllers

- We have learned about sequential and combinational components of datapath.
- Now we are going to cover controller part





Outline

4- Controllers

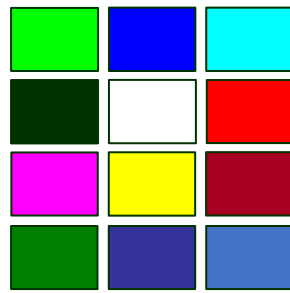
Finite State Machines

4-1 Moore machine

4-2 Mealy machine

4-3 Huffman model

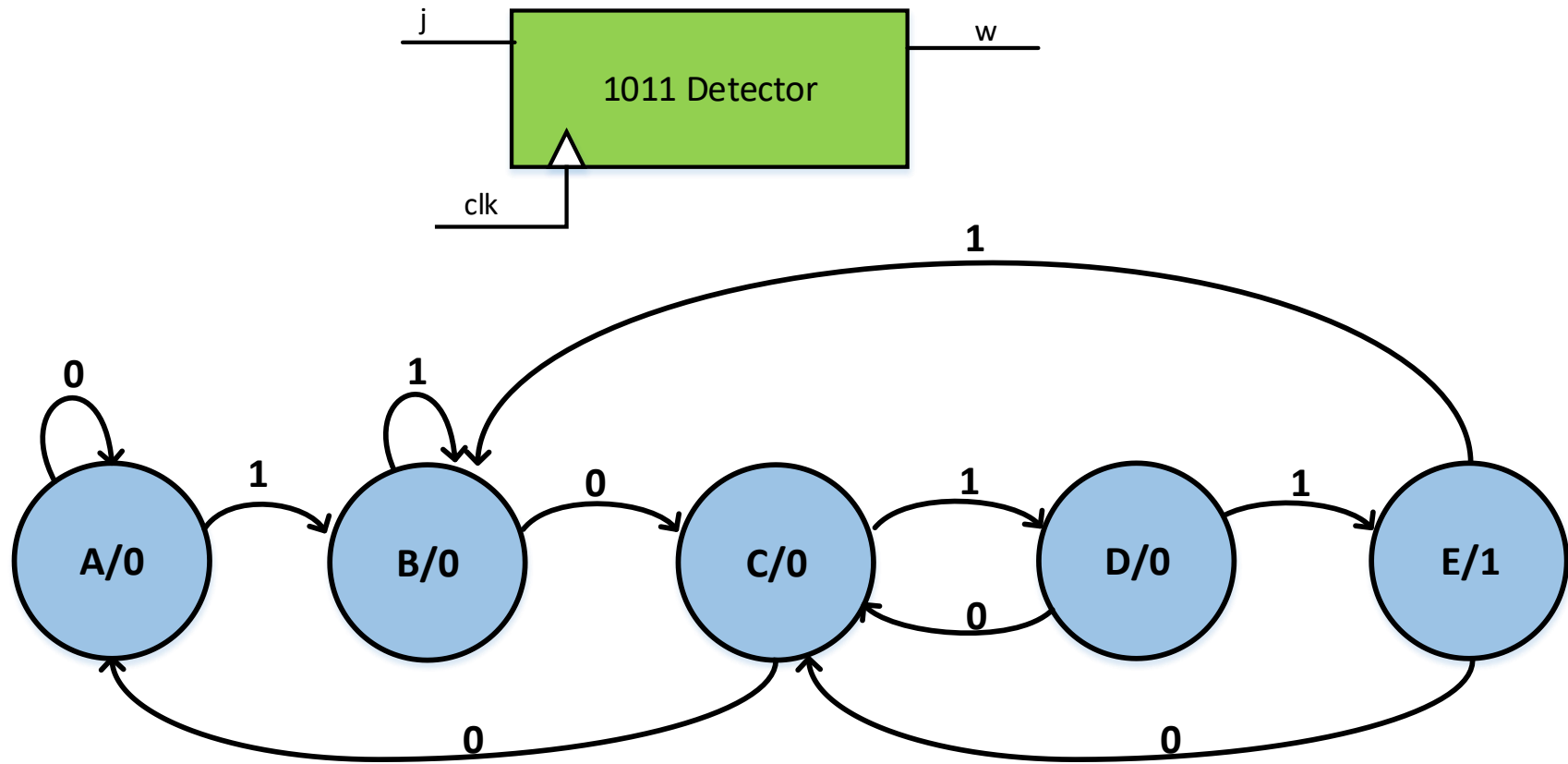
4-4 Hierarchical FSM



Finite State Machine

Moore Machine

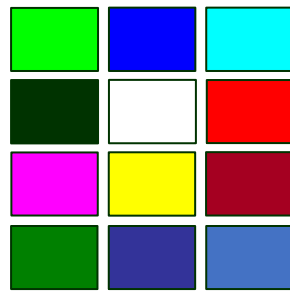
- We start with designing a 1011 sequence detector using Moore machine



State diagram

Finite State Machine

Moore Machine

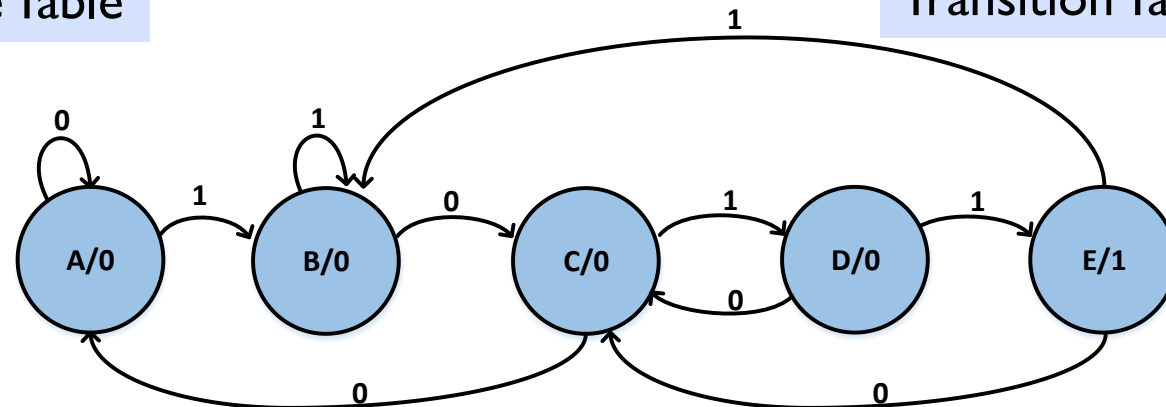


	State	0	1	w
	A	A	B	0
	B	C	B	0
	C	A	D	0
	D	C	E	0
	E	C	B	1
		State ⁺		

State Table

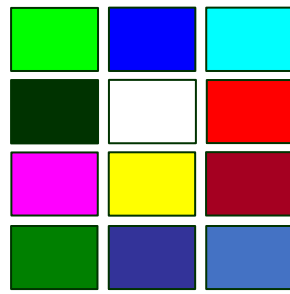
$V_2V_1V_0$	0	1	w
000	000	001	0
001	010	001	0
010	000	011	0
011	010	100	0
100	010	001	1
	$V_2^+V_1^+V_0^+$		

Transition Table



Finite State Machine

Moore Machine

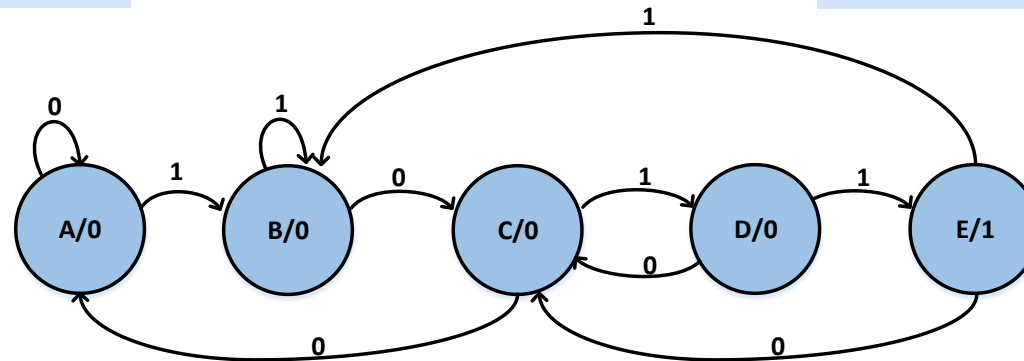


$V_2V_1V_0$	0	1	w
000	000	001	0
001	010	001	0
010	000	011	0
011	010	100	0
100	010	001	1
	$V_2^+V_1^+V_0^+$		

Transition Table

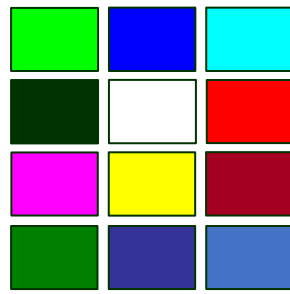
$V_2V_1V_0$	0	1	w
000	000	001	0
001	010	001	0
010	000	011	0
011	010	100	0
100	010	001	1
	$D_2D_1D_0$		

Excitation Table



Finite State Machine

Moore Machine



jV_2 V_1V_0	00	01	11	10
00	0	0	0	0
01	0	-	-	0
11	0	-	-	1
10	0	-	-	0

D_2

$$D_2 = j \cdot V_1 \cdot V_0$$

jV_2 V_1V_0	00	01	11	10
00	0	1	0	0
01	1	-	-	0
11	1	-	-	0
10	0	-	-	1

D_1

$$D_1 = \bar{j} \cdot V_2 + \bar{j} \cdot V_0 + j \cdot V_1 \cdot \bar{V}_0$$

jV_2 V_1V_0	00	01	11	10
00	0	0	1	1
01	0	-	-	1
11	0	-	-	0
10	0	-	-	1

D_0

$$D_0 = j \cdot \bar{V}_1 + j \cdot \bar{V}_0$$

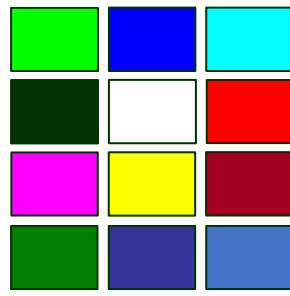
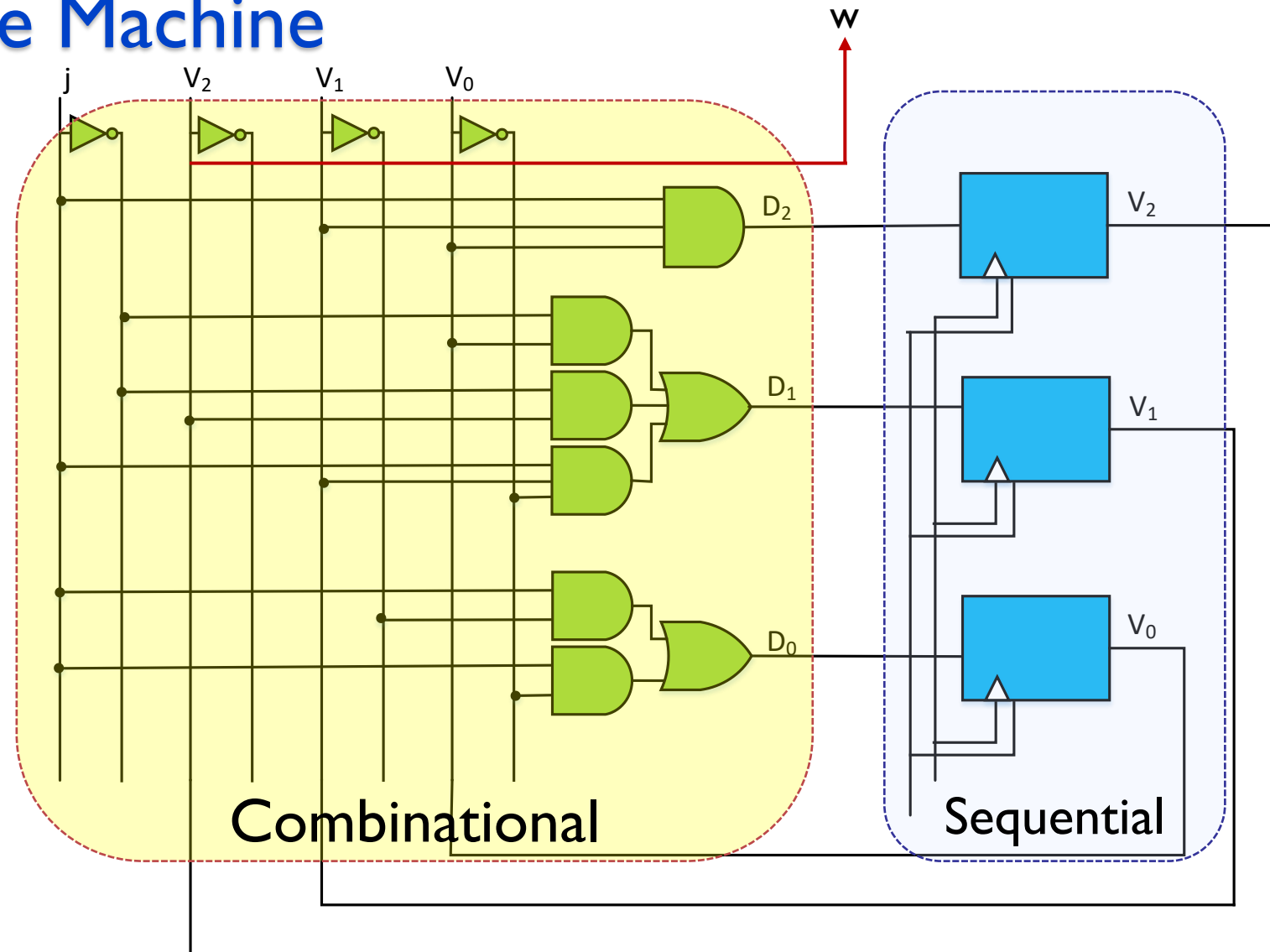
V_2V_1 V_0	00	01	11	10
0	0	0	-	1
1	0	0	-	-

w

$$w = V_2$$

Finite State Machine

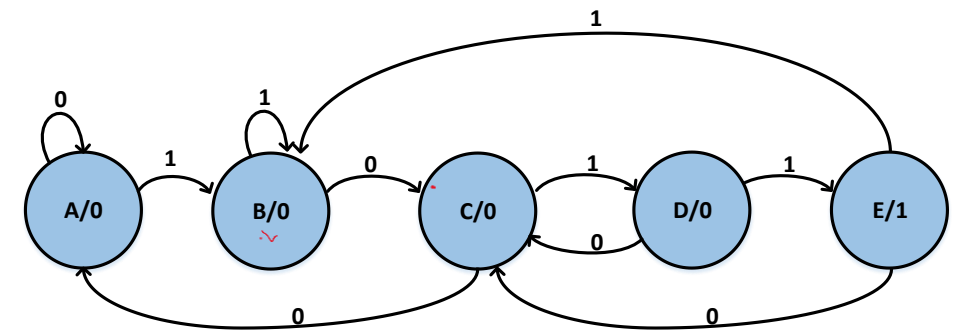
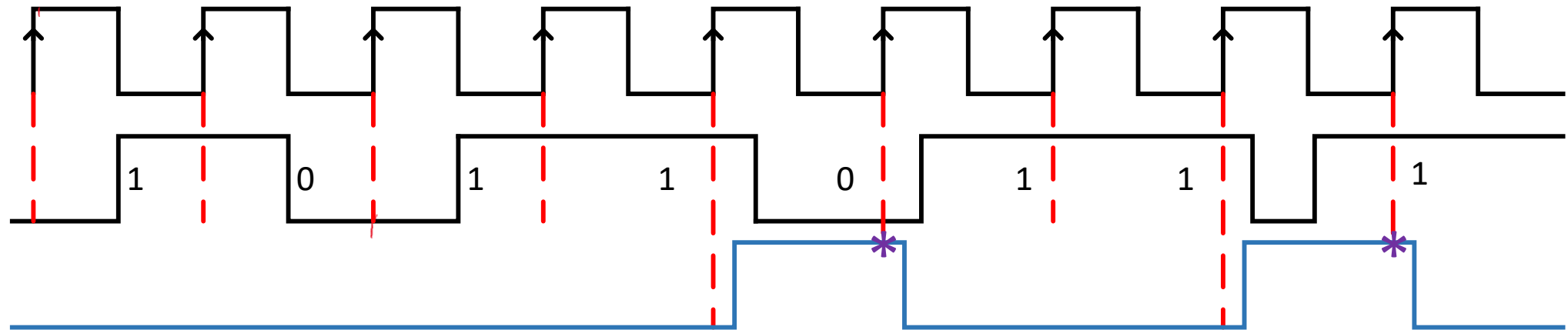
Moore Machine



Finite State Machine

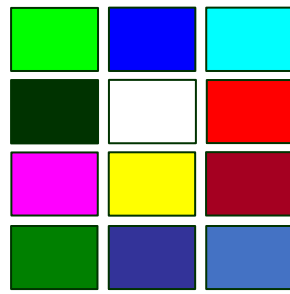
Moore Machine (Timing)

Values are only important on the rising edges of the clock pulses

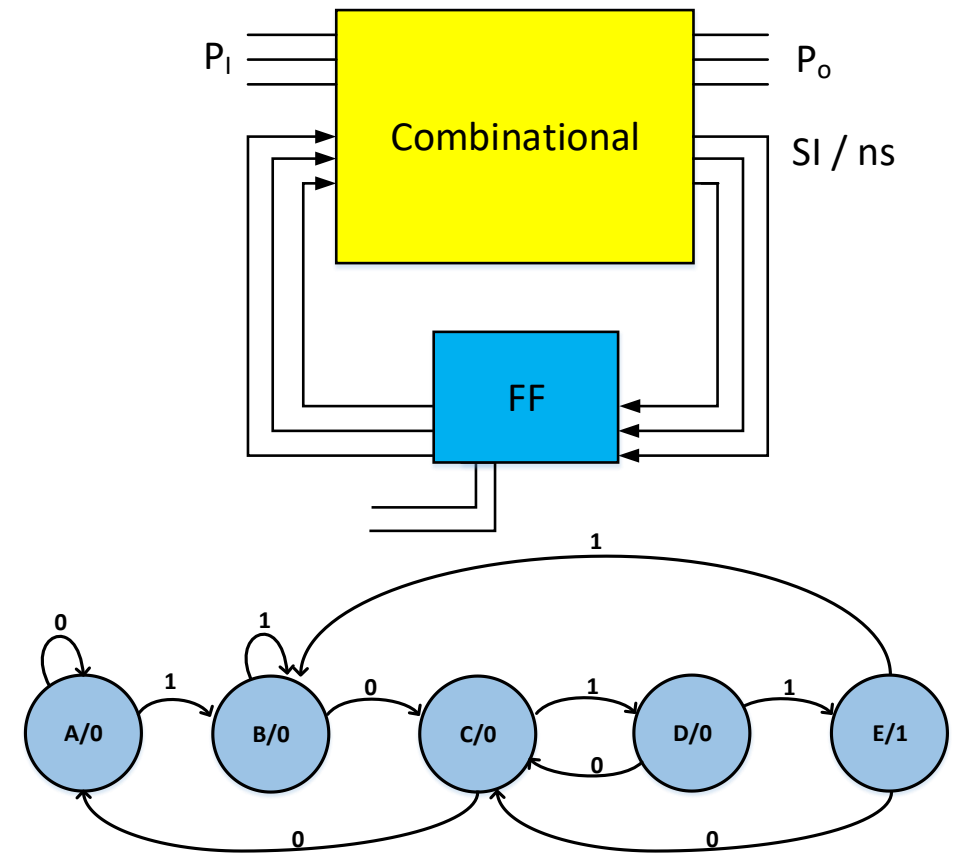


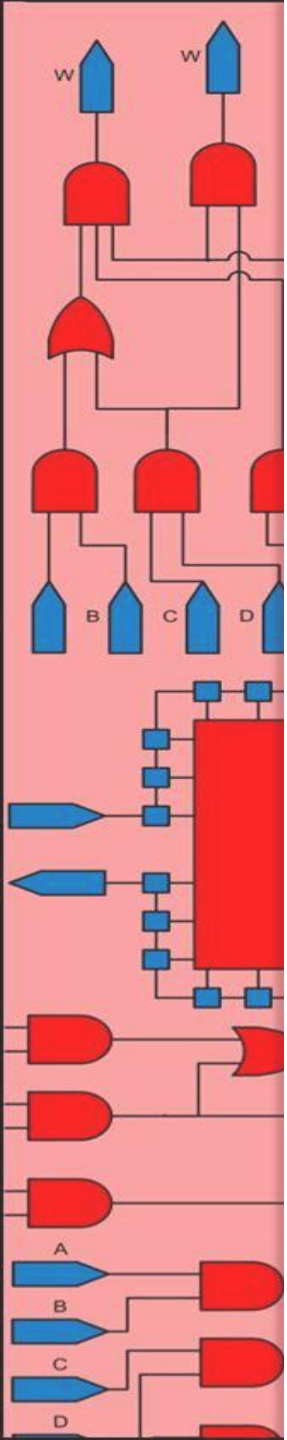
Finite State Machine

Moore Machine (SystemVerilog Description)



```
module moore_1011 (  
    input  logic clk,  
    input  logic rst_n,  
    input  logic j,  
    output logic w  
);  
  
    logic [2:0] next_state, present_state;  
  
    always_comb begin  
        next_state = 3'b000;  
        case(present_state)  
            3'b000: next_state = j ? 3'b001 : 3'b000;  
            3'b001: next_state = j ? 3'b001 : 3'b010;  
            3'b010: next_state = j ? 3'b011 : 3'b000;  
            3'b011: next_state = j ? 3'b100 : 3'b010;  
            3'b100: next_state = j ? 3'b001 : 3'b010;  
            default: next_state = 3'b000;  
        endcase  
    end  
  
    assign w = (present_state == 3'b100) ? 1'b1 : 1'b0;  
  
    always_ff @(posedge clk or negedge rst_n) begin  
        if (!rst_n)  
            present_state <= 3'b000;  
        else  
            present_state <= next_state;  
        end  
    end  
endmodule
```





Outline

4- Controllers

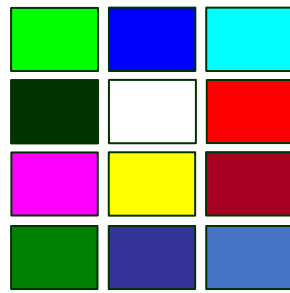
Finite State Machines

4-1 Moore machine

4-2 Mealy machine

4-3 Huffman model

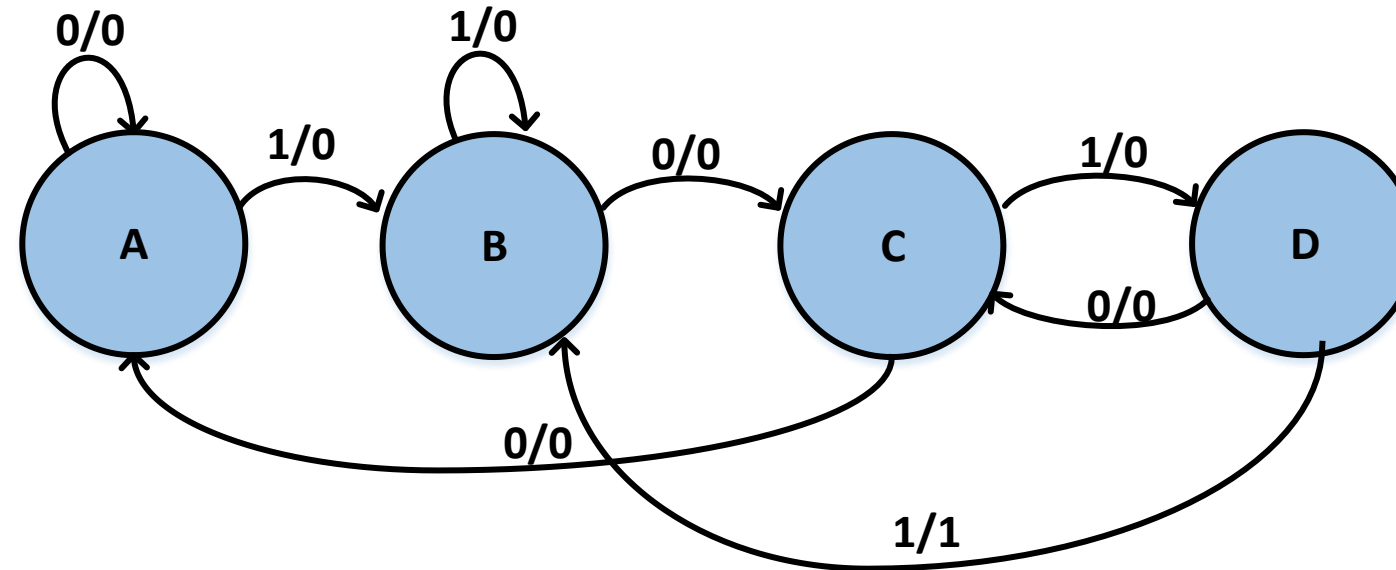
4-4 Hierarchical FSMs



Finite State Machine

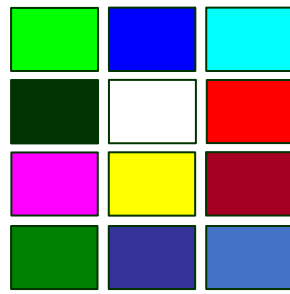
Mealy Machine

- Let's design the same sequence detector with Mealy machine



Finite State Machine

Mealy Machine

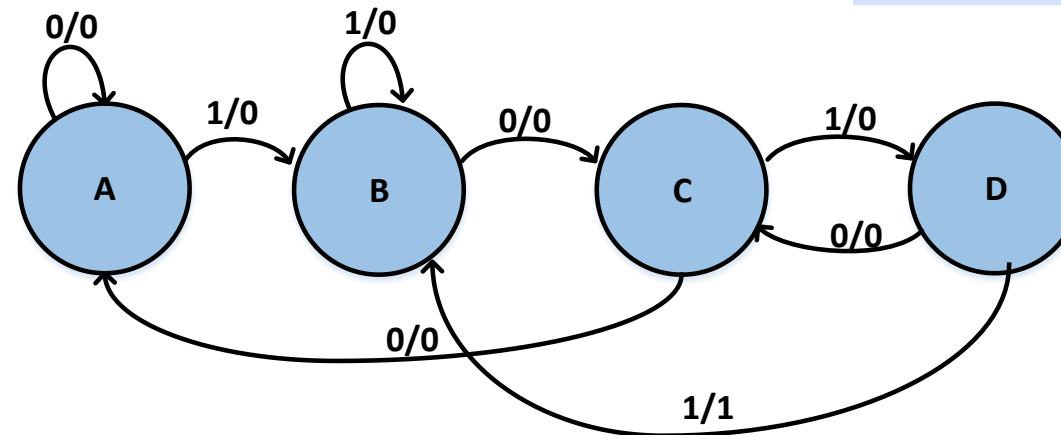


	State	0	1	0	1
	A	A	B	0	0
	B	C	B	0	0
	C	A	D	0	0
	D	C	B	0	1
		State ⁺		w	

State Table

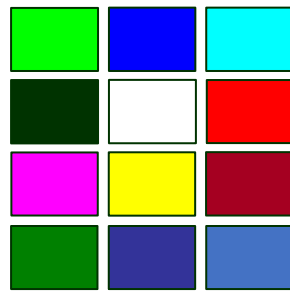
V_1V_0	0	1	0	1
00	00	01	0	0
01	11	01	0	0
11	00	10	0	0
10	11	01	0	1
	$V_1^+V_0^+$		w	

Transition Table



Finite State Machine

Mealy Machine

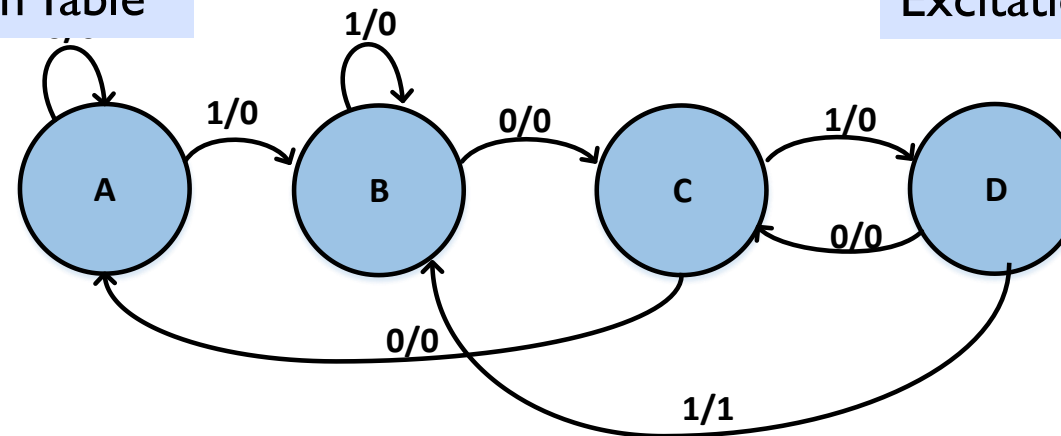


V_1V_0	0	1	0	1
00	00	01	0	0
01	11	01	0	0
11	00	10	0	0
10	11	01	0	1
	$V_1^+V_0^+$		w	

Transition Table

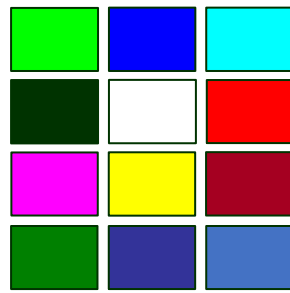
V_1V_0	0	1	0	1
00	00	01	0	0
01	11	01	0	0
11	00	10	0	0
10	11	01	0	1
	D_1D_0		w	

Excitation Table



Finite State Machine

Mealy Machine



$V_1V_0 \backslash j$		0	1
		0	1
00		0	0
01		1	0
11		0	1
10		1	0

D_1

$$D_1 = (V_1 \oplus V_0 \oplus j) \cdot \overline{(j \cdot \overline{V_1})}$$

$V_1V_0 \backslash j$		0	1
		0	1
00		0	1
01		1	1
11		0	0
10		1	1

D_0

$$D_0 = (V_1 \oplus V_0) + j \cdot \overline{V_1}$$

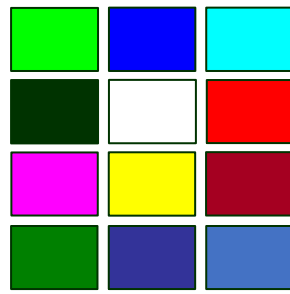
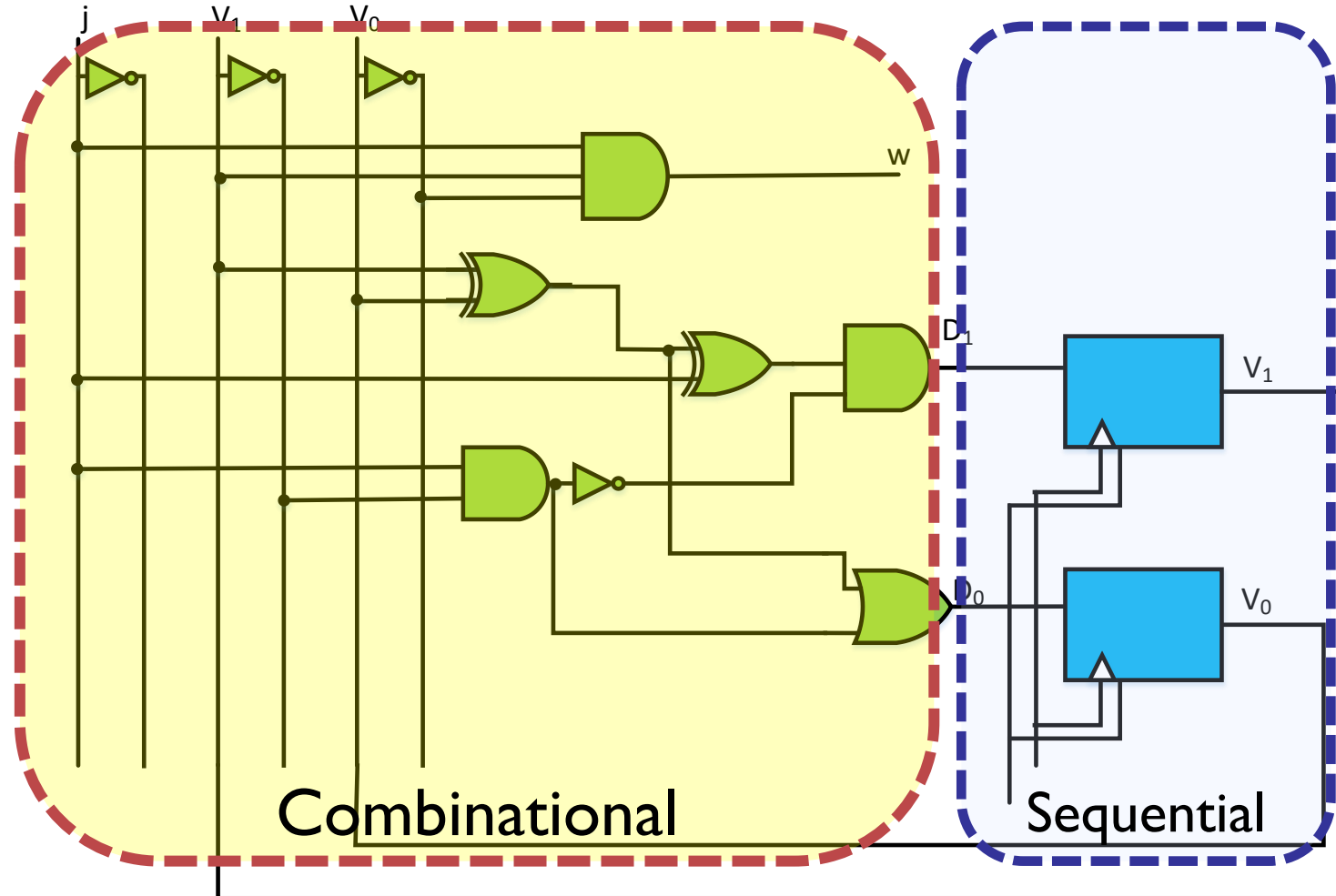
$V_1V_0 \backslash j$		0	1
		0	1
00		0	0
01		0	0
11		0	0
10		0	1

w

$$w = j \cdot V_1 \cdot \overline{V_0}$$

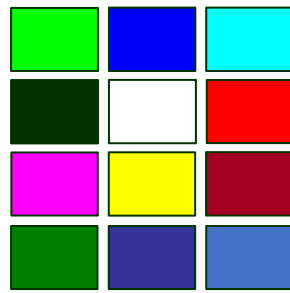
Finite State Machine

Mealy Machine



Finite State Machine

Mealy Machine (SystemVerilog Description)



```

module mealy_1011(
    input  logic J,
    input  logic clk,
    input  logic rst_n,
    output logic W);

    typedef enum logic [1:0] {A, B, C, D} state;

    state p_state, n_state;

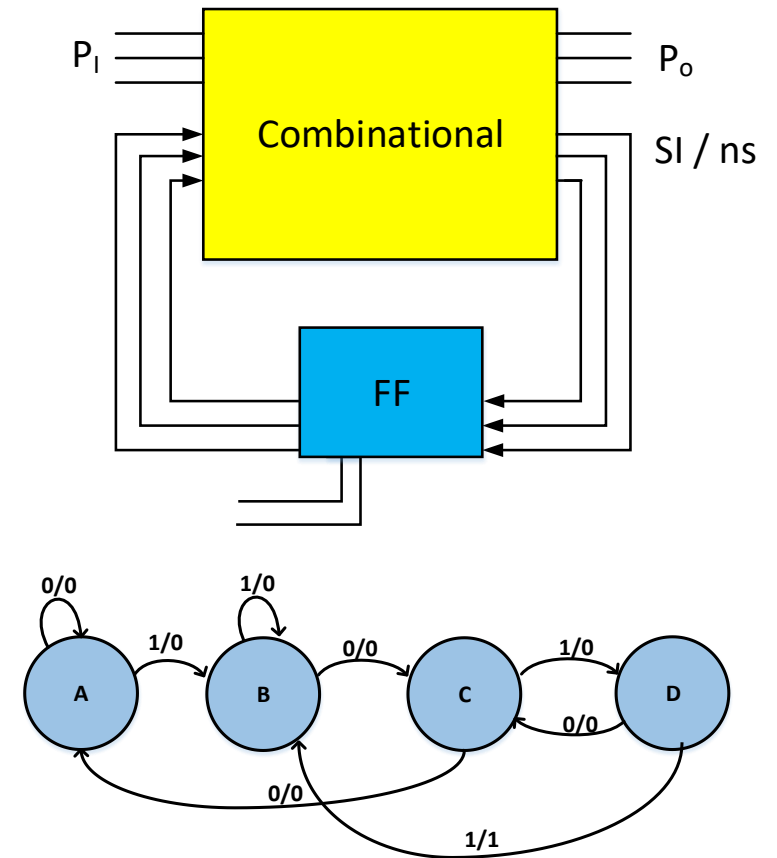
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            p_state <= A;
        else
            p_state <= n_state;
    end

    always_comb begin
        n_state = A;
        case (p_state)
            A: n_state = J ? B : A;
            B: n_state = J ? B : C;
            C: n_state = J ? D : A;
            D: n_state = J ? B : C;
            default: n_state = A;
        endcase
    end

    assign W = (p_state == D) ? J : 1'b0;

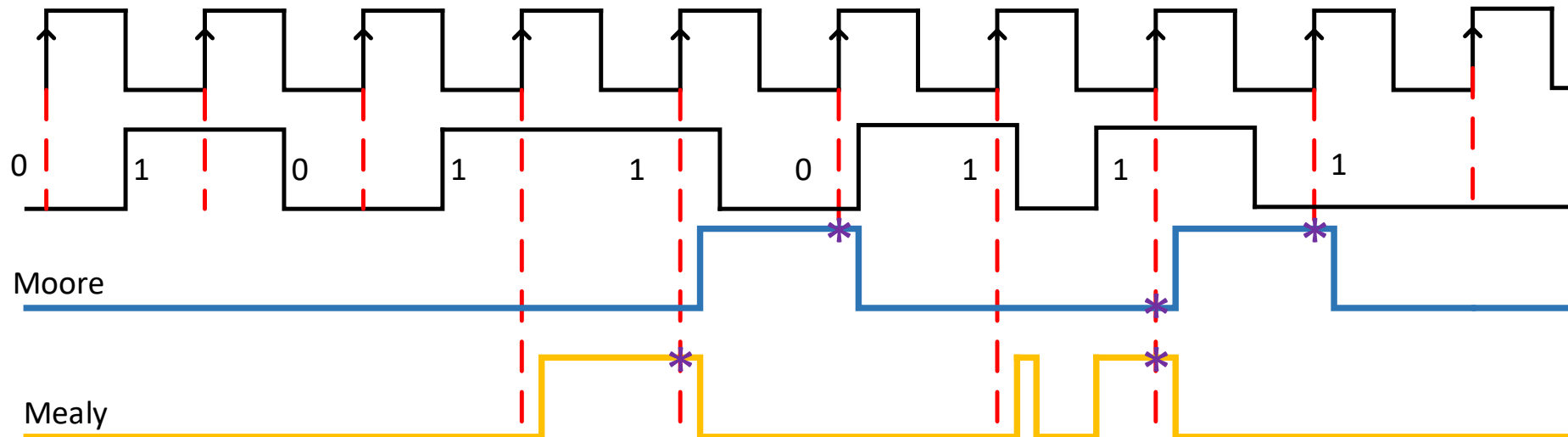
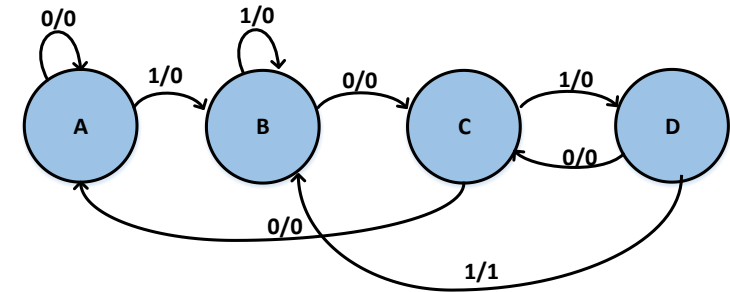
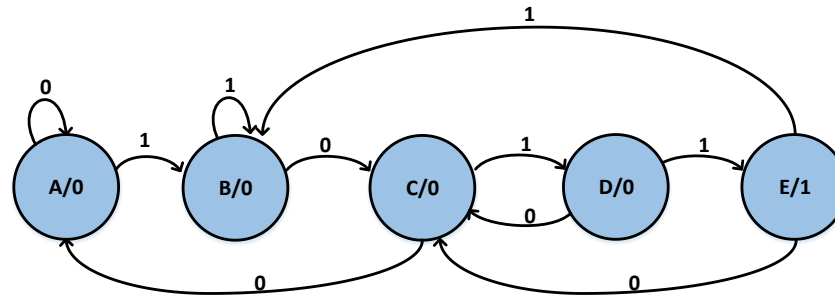
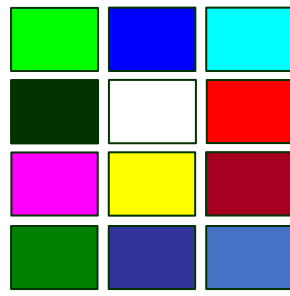
endmodule

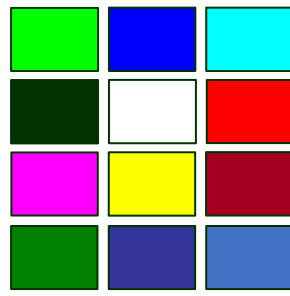
```



Finite State Machine

Mealy Machine vs Moore Machine (Timing)





Finite State Machine

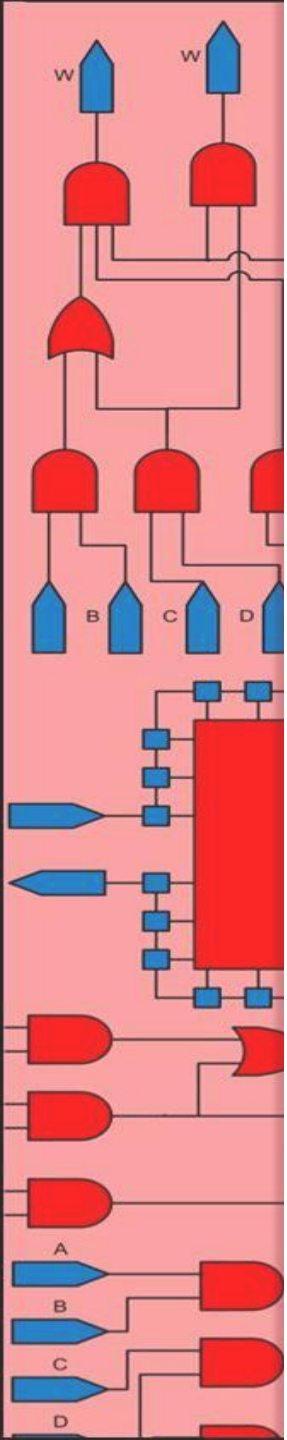
Mealy Machine vs Moore Machine

Moore Machine

1. Output depends on the present state only
2. Input changes between clock don't propagate to the output
3. Generally, it has more states than Mealy
4. It reacts slower to inputs (One clock cycle later)
5. Output is always synchronous with the clock

Mealy Machine

1. Output depends on the present state and present input
2. Input changes can propagate to the output
3. Generally, it has fewer states than Moore Machine
4. They react faster to inputs
5. Asynchronous output generation



Outline

4- Controllers

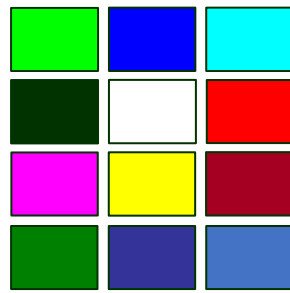
Finite State Machines

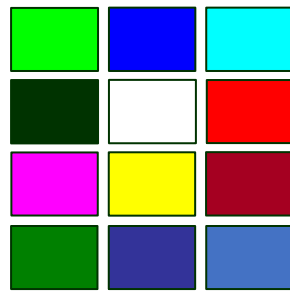
4-1 Moore machine

4-2 Mealy machine

4-3 Huffman model

4-4 Hierarchical FSMs

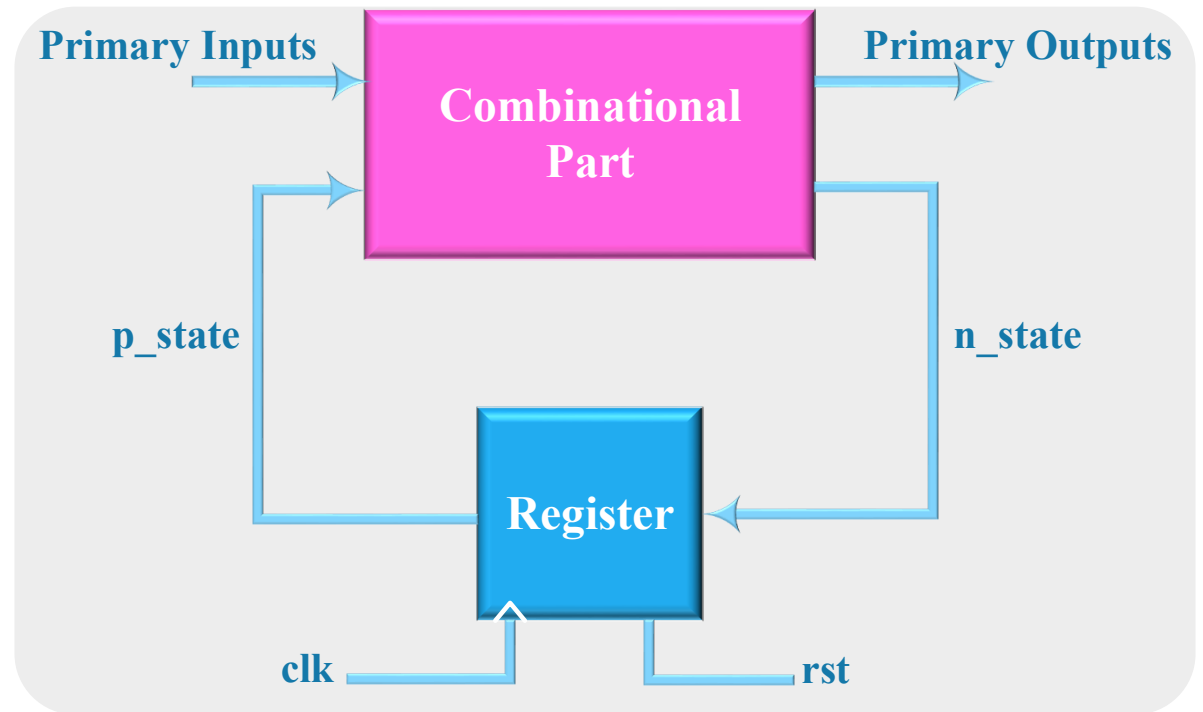


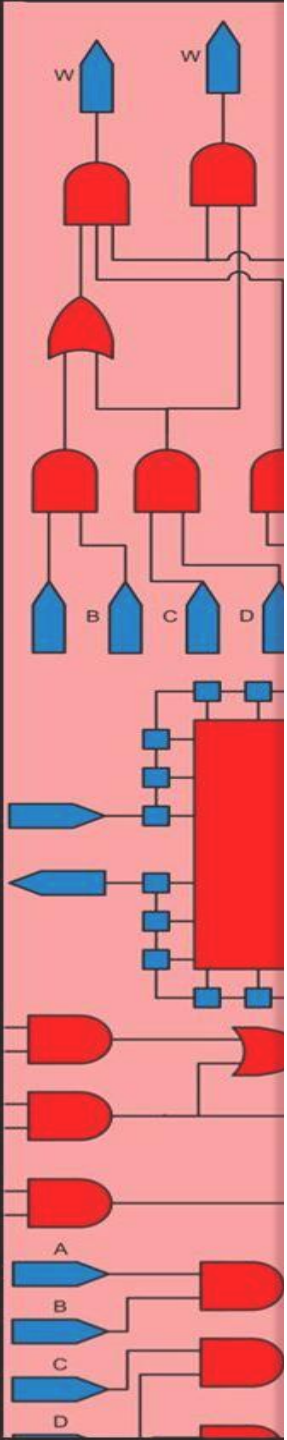


Finite State Machine

Huffman model

- A better coding style for sequential circuit; separates combinational and sequential parts.
- This is called Huffman model.





Outline

4- Controllers

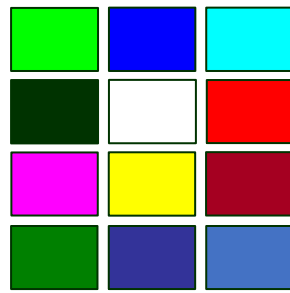
Finite State Machines

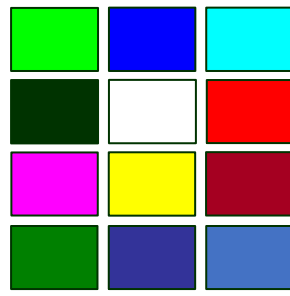
4-1 Moore machine

4-2 Mealy machine

4-3 Huffman model

4-4 Hierarchical FSMs





Finite State Machine

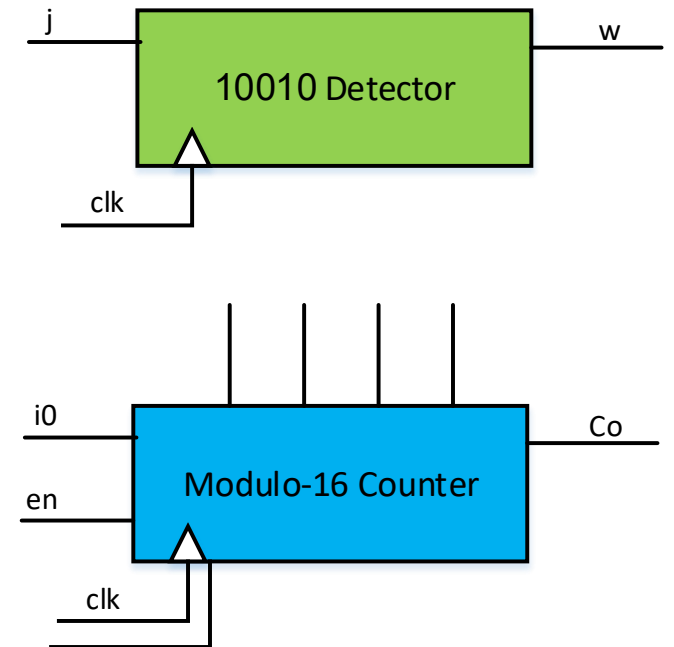
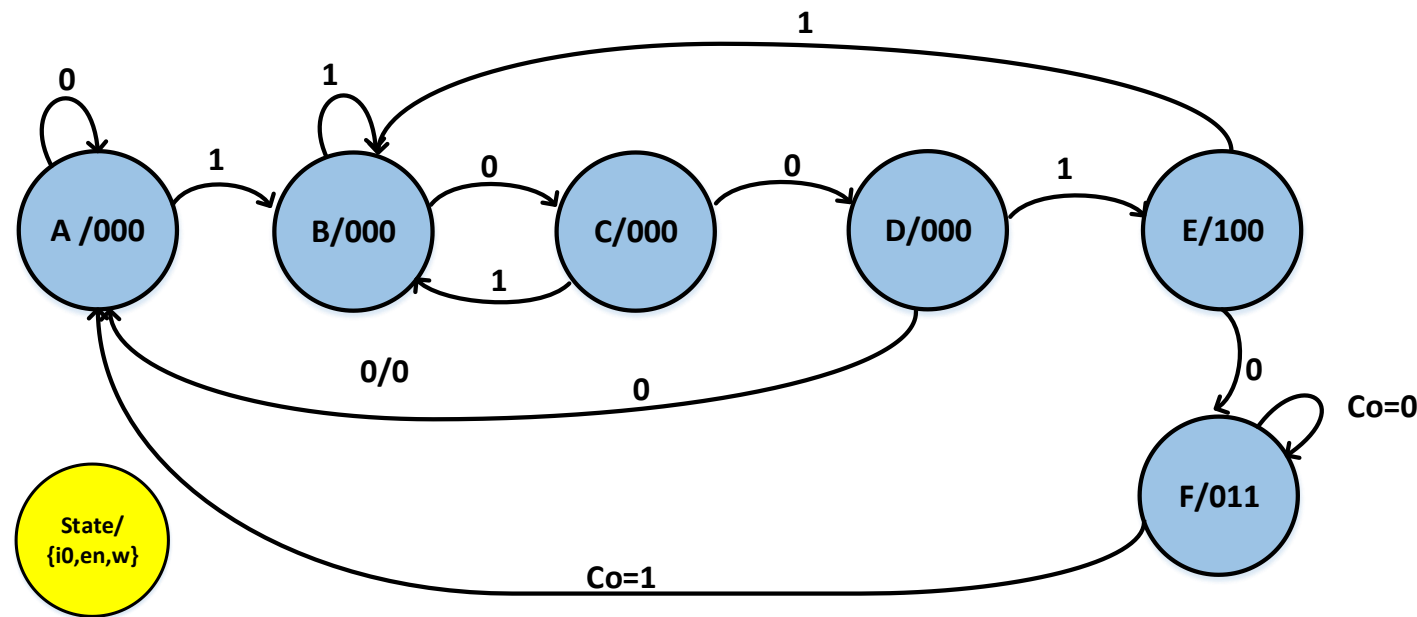
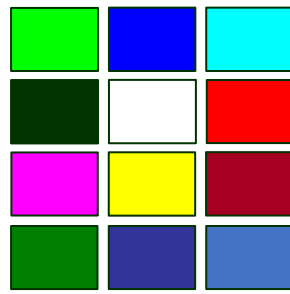
Hierarchical FSMs

Consider this example:

- The machine detects the sequence 10010 on its j input and asserts its w output when the sequence is found.
- After this, the machine waits 16 clock cycles and then begins the search for the next 10010 sequence.
- During these 16 clock cycles, data on the j input are ignored.
- During this time, the w output remains at 1 and becomes 0 as soon as the search begins again.

Finite State Machine

Hierarchical FSMs



Finite State Machine

Hierarchical FSMs

```

module orthogonal_FSMs(
    input  logic J,
    input  logic clk,
    input  logic rst_n,
    output logic W);

typedef enum logic [2:0] {
    A, B, C, D, E, F
} state;

state p_state, n_state;
logic i0, en;
logic [3:0] count;
logic co;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) p_state <= A;
    else p_state <= n_state;
end
    
```

```

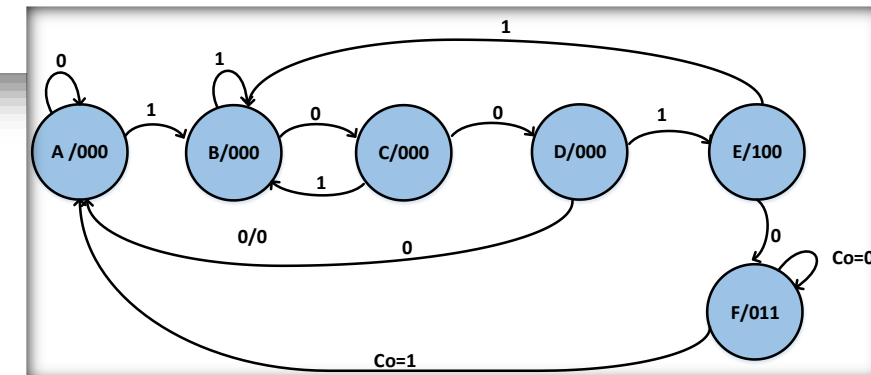
always_comb begin
    n_state = A; i0 = 1'b0; en = 1'b0;

    case (p_state)
        A: n_state = J ? B : A;
        B: n_state = J ? B : C;
        C: n_state = J ? B : D;
        D: n_state = J ? E : A;
        E: begin i0 = 1'b1; n_state = J ? B : F; end
        F: begin en = 1'b1; n_state = co ? A : F; end
        default: begin n_state = A;
                       i0 = 1'b0; en = 1'b0; end
    endcase
end

assign W = (p_state == F) ? 1'b1 : 1'b0;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) count <= 4'b0000;
    else if (i0) count <= 4'b0000;
    else if (en) count <= count + 1'b1;
end

assign co = &count;
endmodule
    
```



Conclusions

Moore machine

Mealy machine

Huffman model

Hierarchical FSMs

